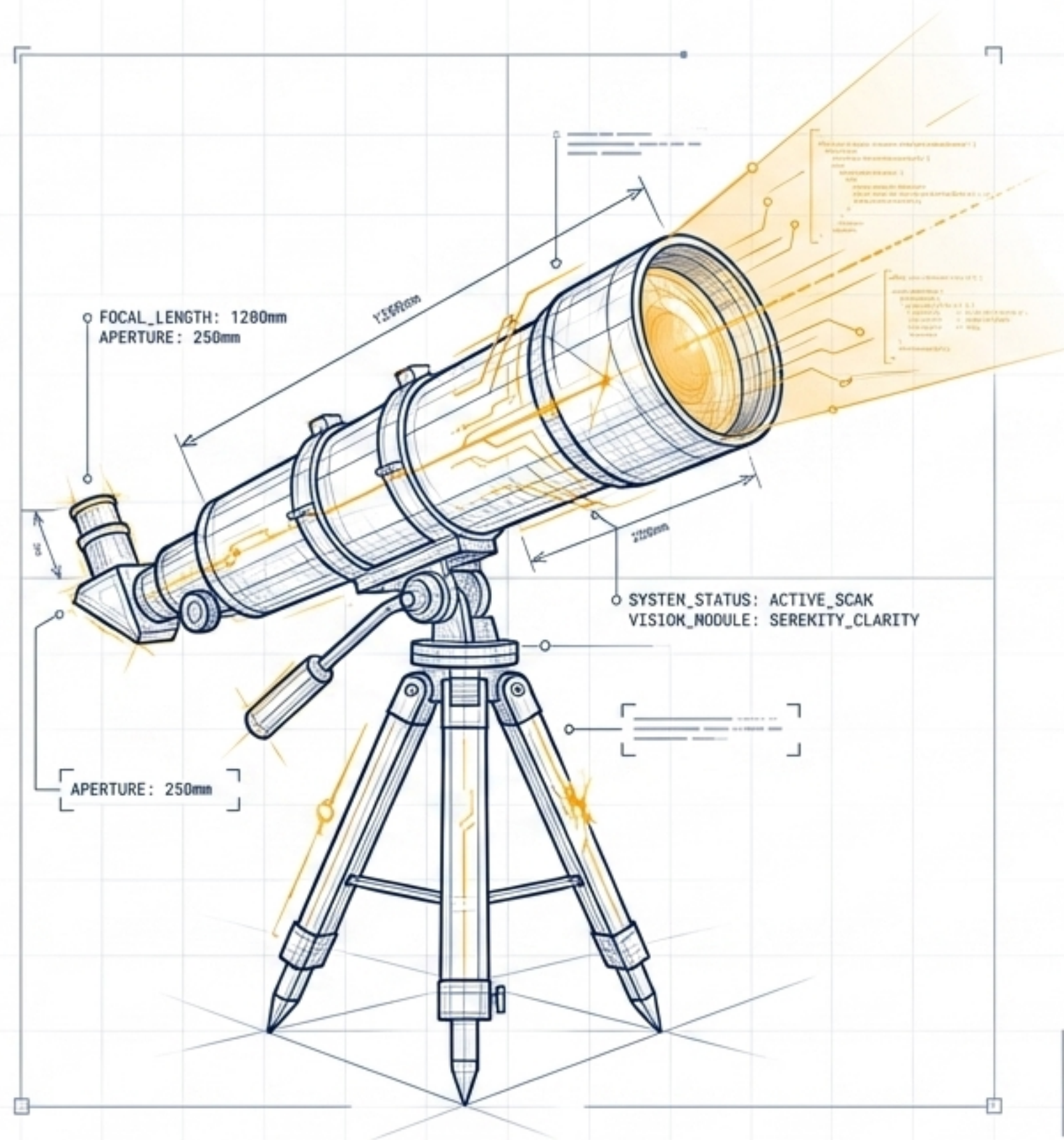


DOSSIÊ ARQUITETURAL

Serenity BDD: Automação com Estilo e Clareza

Transformando testes tradicionais
em documentação viva

Público-Alvo: Seminário de
Engenharia de Software



0 Paradoxo dos Testes Legados



Difíceis de Manter

Testes acoplados à implementação quebram a cada refatoração. Consumo excessivo de tempo do time.



Difíceis de Ler

Falta de semântica clara. Exigem contexto técnico profundo para compreensão, mesmo pelo autor original.



Invisíveis para Stakeholders

Relatórios técnicos não comunicam valor. Gerentes e clientes não conseguem interpretar os resultados do negócio.

A Engenharia do Serenity BDD

Um framework open-source em Java que funde Behavior-Driven Development (BDD) com a geração de relatórios ricos, integrando-se nativamente com JUnit, Cucumber e Maven.



Relatórios Ricos

Geração automática de relatórios visuais detalhados que eliminam a opacidade técnica.



Documentação Viva

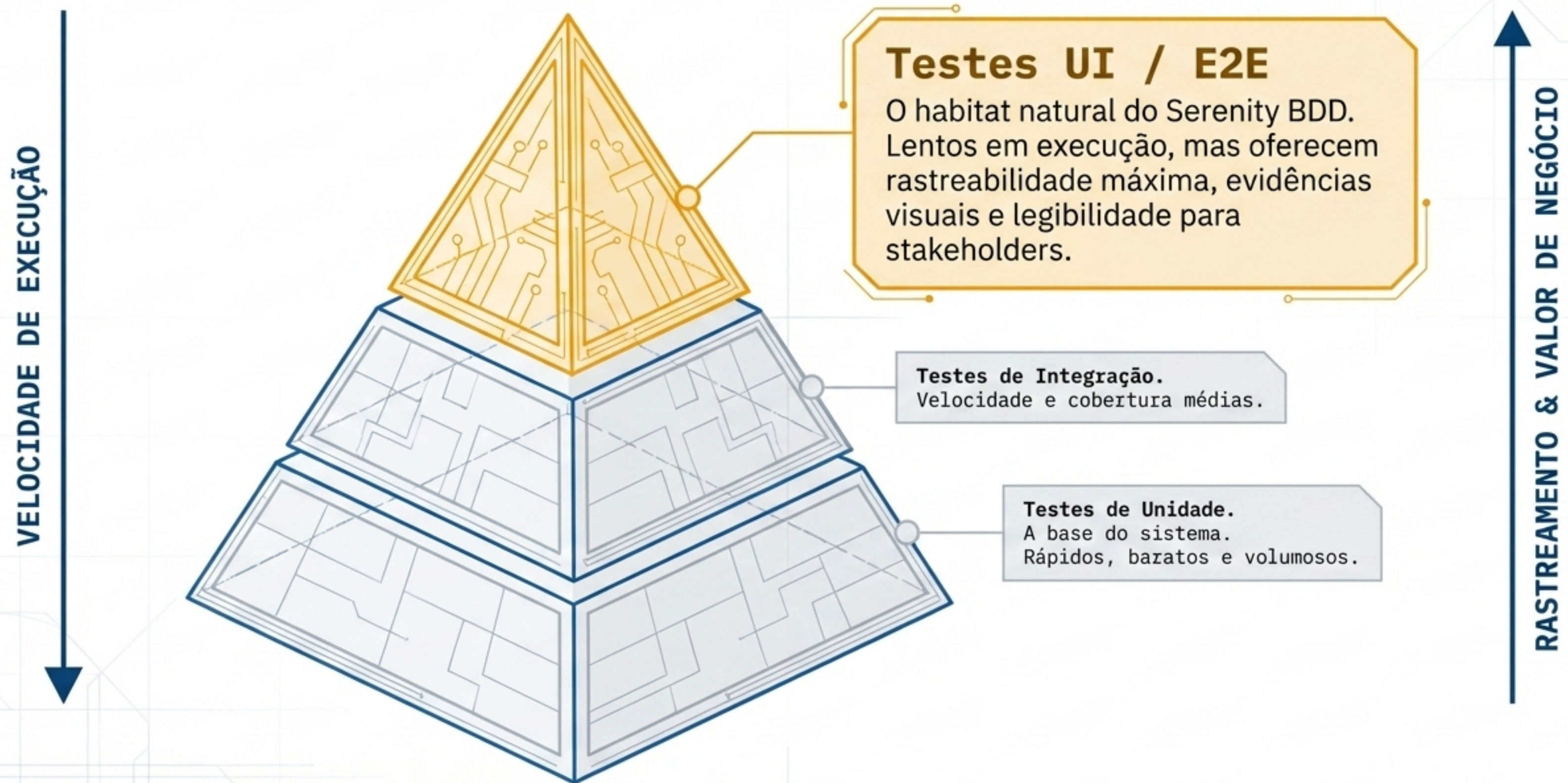
Testes que narram ativamente o comportamento real do sistema, sempre sincronizados com o código.



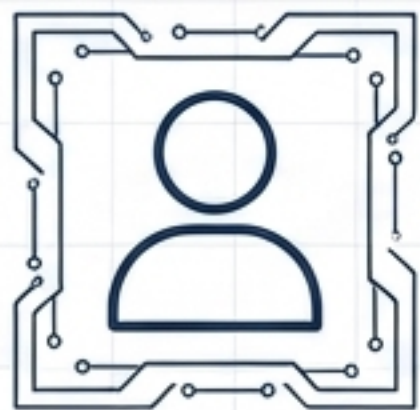
Padrão Screenplay

Uma arquitetura robusta e escalável, orientada a atores, tarefas e perguntas.

Topografia do Sistema: Onde o Serenity Atua



A Arquitetura Screenplay



Atores (**Actors**)

Representam usuários ou sistemas interagindo com a aplicação.

executes



Tarefas (**Tasks**)

Ações de alto nível executadas pelo ator.

asks



Perguntas (**Questions**)

Verificações rigorosas sobre o estado do sistema.

IDE

```
Actor joao = Actor.named("João");
joao.attemptsTo(
    Login.withCredentials("joao@email.com", "senha123")
);
assertThat(joao.asksAbout(TheHomePage.title()))
    .isEqualTo("Bem-vindo, João!");
```

O Pipeline da Documentação Viva



O Negócio (Gherkin)

Cenário: Login com credenciais válidas
Dado que João está na página de login
Quando ele informa o e-mail "joao@email.com"
E a senha "senha123"
E clica em "Entrar"
Então ele deve ver a mensagem "Bem-vindo, João!"



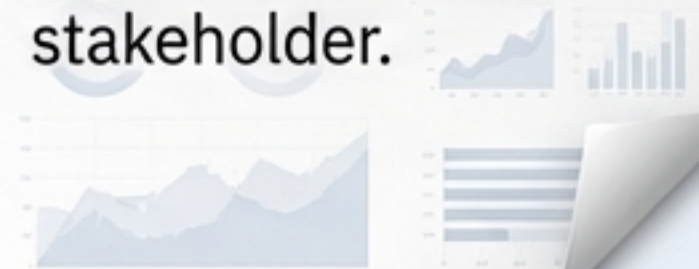
A Execução (Screenplay / Java)

```
@Given("que João está na  
página de login")  
public void setup() {  
    joao.wasAbleTo(NavigateTo.  
theLoginPage());  
}  
  
@When("ele informa o e-mail  
{string}")  
public void login(String  
email) {  
    joao.attemptsTo(Enter.theVa  
lue(email).into(LOGIN_FIELD));  
}
```

O Resultado (Relatório)

Evidência Visual

O código traduz a linguagem natural em passos rastreáveis legíveis por qualquer stakeholder.



Diagnóstico de Mercado: Serenity vs Alternativas



	Serenity	Cypress	Playwright
Linguagem	Java	JavaScript / TypeScript	JS / TS / Python / Java / C#
Foco Principal	BDD, relatórios ricos e rastreabilidade ponta a ponta.	Testes frontend rápidos com debugging visual em tempo real.	Automação cross-browser moderna, rápida e flexível.
Curva de Aprendizado	Alta (exige domínio de Screenplay + Gherkin).	Baixa (API altamente intuitiva).	Média (API robusta e bem documentada).
Cenário de Domínio (Melhor para)	Projetos enterprise Java que exigem auditoria por stakeholders não técnicos.	SPAs e times de frontend necessitando de ciclo de feedback rápido.	Times que precisam de suporte multi-browser agressivo e multi-linguagem.

Análise de Trade-offs

Relatórios Visuais

Evidências automáticas com screenshots, logs e narrativa executada.

Documentação Viva

Reflete os requisitos reais, sempre atualizada com o código.

Integração Nativa

Pipelines de CI/CD suportados imediatamente (JUnit, Maven, Gradle).



! Curva Abstrata

O padrão **Screenplay** exige uma mudança brusca de mentalidade em relação ao Page Objects clássico.

! Configuração Complexa

Setup de dependências (Maven/Gradle), drivers e plugins exige esforço inicial alto.

! Testes Lentos

Não substitui a base da pirâmide; execuções E2E são inerentemente lentas.

Matriz de Decisão: Quando Adotar?

Cenário Ideal (GO)



O projeto é um sistema Java Enterprise com longa vida útil prevista.



Há stakeholders ou clientes não técnicos que precisam consumir resultados operacionais.



Rastreabilidade sistêmica e documentação viva são requisitos inegociáveis.



O time já possui maturidade e cultura estabelecida em BDD e Cucumber.

Cenário Incompatível (NO-GO)



O projeto é uma startup ágil baseada em stack frontend (React, Vue, etc.).



O time é reduzido e exige um loop de feedback ultra-rápido no desenvolvimento.



Não existe cultura estabelecida de escrita em Gherkin ou BDD.



A janela de prazo inviabiliza o overhead de configuração inicial do framework.

O Veredito Estratégico



1. Screenplay como Arquitetura

Atores, Tarefas e Perguntas forçam um design expressivo, altamente reutilizável e estritamente orientado ao comportamento.



2. Testes como Documentação

Cada execução gera um artefato de comunicação vivo, unificando a engenharia de software e a auditoria de negócios.



3. Ferramenta de Nicho, Não Bala de Prata

O Serenity brilha em projetos Java maduros, mas pune times sem forte cultura e comprometimento com BDD.

Seus testes hoje explicam o que o sistema faz — ou só dizem se passou ou falhou?